

Resolução do Desafio Técnico - Looqbox

Candidato: Déborah Martins

Link do PR:

Olá! Segue abaixo a documentação das soluções que desenvolvi para os 3 cases do desafio, junto com os códigos e as visualizações geradas.

Case 1: Função de Extração

Para essa função, evitei trazer a tabela inteira para a memória do Pandas, já que isso causaria problemas de performance e sobrecarga caso a base de vendas seja muito grande. A abordagem que adotei foi usar consultas parametrizadas para enviar os filtros diretamente para o banco de dados. Assim, o banco faz o trabalho pesado e o Python recebe apenas os dados já filtrados e seguros contra injeção de SQL.

```
import pandas as pd
from sqlalchemy import create_engine

# Conexão com o banco de dados MySQL
engine = create_engine(

"mysql+pymysql://looqbox-challenge:looq-challenge@35.199.115.174:3306/looqbox-challenge"

)

def retrieve_data(
    product_code: int,
    store_code: int,
    date: list[str]
) -> pd.DataFrame:

    # Validação dos códigos
    if not isinstance(product_code, int):
        raise TypeError("product_code deve ser um inteiro.")

    if not isinstance(store_code, int):
        raise TypeError("store_code deve ser um inteiro.")

    # Validação das datas
    if not isinstance(date, list):
        raise TypeError("date deve ser uma lista.")
```

```

if len(date) != 2:
    raise ValueError(
        "date deve conter uma data inicial e uma data final."
    )

try:
    start_date = pd.to_datetime(date[0])
    end_date = pd.to_datetime(date[1])
except ValueError:
    raise ValueError(
        "As datas devem estar em um formato válido, como
YYYY-MM-DD."
    )

if start_date > end_date:
    raise ValueError(
        "A data inicial não pode ser maior que a data final."
    )

# Carga dos dados
query = """
SELECT *
FROM data_product_sales
WHERE PRODUCT_CODE = %(product_code)s
      AND STORE_CODE = %(store_code)s
      AND DATE >= %(start_date)s
      AND DATE <= %(end_date)s
"""

params = {
    "product_code": product_code,
    "store_code": store_code,
    "start_date": date[0],
    "end_date": date[1]
}

df = pd.read_sql(query, engine, params=params)

# Padronização dos dados de origem
df["STORE_CODE"] = pd.to_numeric(df["STORE_CODE"], errors="raise")
df["PRODUCT_CODE"] = pd.to_numeric(df["PRODUCT_CODE"])
df["DATE"] = pd.to_datetime(df["DATE"])

```

```

        return df

# Exemplo de utilização
df_resultado = retrieve_data(18, 1, ["2019-01-01", "2019-01-31"])
print(df_resultado.head())

```

Case 2: Ticket Médio

Seguindo a instrução de manter as queries originais intactas, fiz a extração bruta e apliquei o filtro do último trimestre de 2019 diretamente no Pandas. Para que o Ticket Médio (TM) não sofresse distorções matemáticas, fiz o agrupamento somando o valor e a quantidade totais primeiro, para só depois realizar a divisão.

```

import pandas as pd
from sqlalchemy import create_engine
import matplotlib.pyplot as plt

# Conexão com o banco de dados MySQL
engine = create_engine(

"mysql+pymysql://looqbox-challenge:looq-challenge@35.199.115.174:3306/looqbox-challenge"
)

# Consulta 1 - Cadastro das lojas
query_1 = """
SELECT
    STORE_CODE,
    STORE_NAME,
    START_DATE,
    END_DATE,
    BUSINESS_NAME,
    BUSINESS_CODE
FROM data_store_cad
"""

df_store = pd.read_sql(query_1, engine)

# Consulta 2 - Vendas das lojas
query_2 = """
SELECT
    STORE_CODE,

```

```

        DATE,
        SALES_VALUE,
        SALES_QTY
FROM data_store_sales
WHERE DATE BETWEEN '2019-01-01' AND '2019-12-31'
"""

df_sales = pd.read_sql(query_2, engine)

# Tratamento dos dados
df_sales["DATE"] = pd.to_datetime(df_sales["DATE"])

# Filtrando os dados de vendas para o período de 01/10/2019 a
31/12/2019
start_date = "2019-10-01"
end_date = "2019-12-31"

df_sales_filtrado = df_sales[
    (df_sales["DATE"] >= start_date) &
    (df_sales["DATE"] <= end_date)
]

# print(df_sales_filtrado)

# Relacionando os dados de vendas com os dados de cadastro das lojas
df = df_sales_filtrado.merge(
    df_store,
    on="STORE_CODE",
    how="left"
)

# print(df)
# print(df[['STORE_NAME', 'BUSINESS_NAME']])

# Cálculo do TM
df_result = (
    df.groupby(
        ["STORE_NAME", "BUSINESS_NAME"],
        as_index=False
    )
    .agg(
        SALES_VALUE=("SALES_VALUE", "sum"),
        SALES_QTY=("SALES_QTY", "sum")
    )

```

```

    )
)

df_result["TM"] = df_result["SALES_VALUE"] / df_result["SALES_QTY"]

# Selecionando as colunas
df_result = df_result[
    ["STORE_NAME", "BUSINESS_NAME", "TM"]
]

# Renomeando as colunas
df_result = df_result.rename(
    columns={
        "STORE_NAME": "Loja",
        "BUSINESS_NAME": "Categoria",
    }
)

df_result["TM"] = df_result["TM"].round(2)
print(df_result)

# Cópia para formatação do TM com vírgula
df_bras = df_result.copy()
df_bras["TM"] = df_bras["TM"].map(
    lambda x: f"{x:.2f}".replace(".", ",")
)

# Criando tabela visual com Matplotlib
fig, ax = plt.subplots(figsize=(8, 8))
ax.axis("off")

# Criando a tabela
table = ax.table(
    cellText=df_bras.values,
    colLabels=df_bras.columns,
    loc="center",
    cellLoc="center",
    bbox=[0, 0, 1, 0.9]
)

table.auto_set_font_size(False)
table.set_fontsize(11)

```

```
# Pintando o cabeçalho para dar destaque
for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_text_props(weight='bold', color='white')
        cell.set_facecolor('#1f77b4')

plt.title(
    "Ticket Médio (TM) por Loja e Categoria\n(Out - Dez / 2019)",
    fontsize=14,
    fontweight="bold",
    y=0.95
)

plt.tight_layout()
plt.show()
```

**Ticket Médio (TM) por Loja e Categoria
(Out - Dez / 2019)**

Loja	Categoria	TM
Bahia	Atacado	15,39
Bangkok	Posto	13,67
Belem	Proximidade	15,37
Berlin	Proximidade	15,39
Buenos Aires	Atacado	15,39
Chicago	Varejo	15,53
Dubai	Atacado	15,39
Hong Kong	Farma	26,35
London	Farma	28,99
Madri	Farma	29,03
Miami	Posto	13,67
New York	Proximidade	15,39
Paris	Proximidade	15,39
Rio de Janeiro	Farma	29,59
Roma	Varejo	15,39
Salvador	Atacado	15,39
Sao Paulo	Varejo	15,39
Sidney	Posto	13,67
Tokio	Varejo	15,39
Vancouver	Posto	13,67

Case 3: Visualização (IMDB)

Aproveitei a base de filmes para criar duas análises gráficas focadas em remover ruído visual e trazer insights diretos, usando Seaborn e Matplotlib.

1. Engajamento vs. Nota: A ideia foi cruzar o volume de votos com a nota do público. O gráfico mostra que filmes com um volume gigantesco de engajamento raramente têm notas muito baixas, resultando em uma correlação positiva moderada.

```
import pandas as pd
from sqlalchemy import create_engine
import matplotlib.pyplot as plt
```

```

import seaborn as sns

# Visual mais limpo
sns.set_theme(style="whitegrid")

# Conexão e Extração
engine = create_engine(

"mysql+pymysql://looqbox-challenge:looq-challenge@35.199.115.174:3306/looqbox-challenge"

)

query_imdb = "SELECT * FROM IMDB_movies"
df_imdb = pd.read_sql(query_imdb, engine)

# print(df_imdb.shape)
# print(df_imdb.columns.tolist())
# print(df_imdb.head())
# print(df_imdb.info())

# Transformação e Cálculo
df_1 = df_imdb[["Title", "Rating", "Votes", "RevenueMillions", "Metascore"]].copy()
correlation_votes = df_1["Votes"].corr(df_1["Rating"])

# Visualização de Dados
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df_1,
    x="Votes",
    y="Rating",
    alpha=0.5,
    color="#1f77b4",
    edgecolor="w",
    linewidth=0.5
)

# Ajustes de Escala e Rótulos
plt.xscale("log")
plt.xlabel("Volume de Votos (Escala Logarítmica)", fontsize=11, labelpad=10)
plt.ylabel("Avaliação do Público (Rating IMDB)", fontsize=11, labelpad=10)

```

```

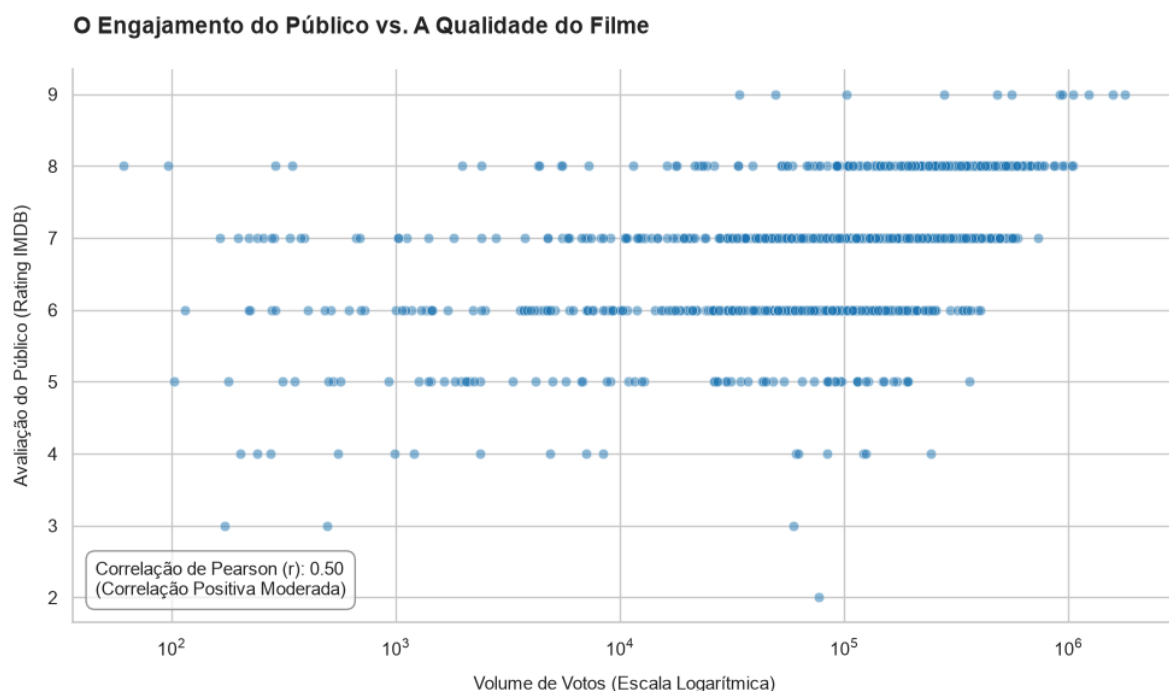
# Título
plt.title("O Engajamento do Público vs. A Qualidade do Filme\n",
          fontsize=14, fontweight="bold", loc="left")

# Anotação da Correlação
plt.annotate(
    f"Correlação de Pearson (r): {correlation_votes:.2f}\n(Correlação  
Positiva Moderada)",
    xy=(0.02, 0.05),
    xycoords='axes fraction',
    fontsize=11,
    bbox=dict(boxstyle="round,pad=0.5", fc="white", ec="gray",
alpha=0.8)
)

# Removendo as bordas superior e direita
sns.despine()

plt.tight_layout()
plt.show()

```



2. Retorno Financeiro por Gênero: Para analisar as receitas, precisei usar o `.explode()` no Pandas, já que muitos filmes carregam vários gêneros na mesma célula. Separando

isso, fica nítido que gêneros com apelo familiar (Animação e Aventura) puxam a média de bilheteria muito para cima em comparação aos demais.

```
# ----- SEGUNDA ANÁLISE: RECEITA MÉDIA POR GÊNERO -----

# Cópia do dataframe e removendo filmes sem dados de receita
df_2 = df_imdb.dropna(subset=['RevenueMillions']).copy()

df_2['Genre_List'] = df_2['Genre'].str.split(',')
df_expl = df_2.explode('Genre_List')

# print(df_expl['Genre_List'].unique())

# Agregação de Dados
df_top_generos = (
    df_expl.groupby('Genre_List')['RevenueMillions']
    .mean()
    .sort_values(ascending=False)
    .head(10) # top10
    .reset_index()
)

# Visualização de Dados
plt.figure(figsize=(10, 6))

barplot = sns.barplot(
    data=df_top_generos,
    x='RevenueMillions',
    y='Genre_List',
    palette='Blues_r'
)

plt.xlabel("Receita Média (Milhões de USD)", fontsize=11, labelpad=10)
plt.ylabel("Gênero Cinematográfico", fontsize=11, labelpad=10)
plt.title("Top 10 Gêneros Cinematográficos Mais Lucrativos\n",
          fontsize=14, fontweight="bold", loc="left")

# Adicionando os valores exatos no final de cada barra
for index, value in enumerate(df_top_generos['RevenueMillions']):
    plt.text(value + 2, index, f"${value:.1f}M", va='center',
             fontsize=10, color='black')

sns.despine(bottom=True)
```

```
plt.xticks(visible=False)

plt.tight_layout()

plt.show()
```

